**Министерство науки и высшего образования Российской Федерации**
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**
**«Национальный исследовательский университет ИТМО»**
**(Университет ИТМО)**

**Факультет прикладной информатики**
**Направление подготовки** 09.03.03 Прикладная информатика
**Образовательная программа** Мобильные и сетевые технологии

# О Т Ч Е Т
## по лабораторной работе №2
«Метод ломаных»

**Обучающийся:** Заботкина Марина Александровна, K3339
**Поток:** МетОпт 1.2

Санкт-Петербург,
2026

## Постановка задачи

Разработать программу на языке Python, решающую задачу поиска глобального экстремума для заданной одномерной Липшицевой функции на заданном отрезке методом ломаных.

На вход программа принимает строку одномерной функции, координаты концов отрезка (вещественные числа), точность вычисления eps. На выходе она выполняет визуализацию графика исходной функции, вспомогательных функций (по возможности итоговой ломаной, на которой достигнута требуемая точность), приближенное значение аргумента и минимального значения функции, число итераций, потраченное время.

В качестве тестовой задачи используется одномерный срез функции Растригина:

$$W(x) = x^2 - 10\cos(2\pi x) + 10, \quad x \in [-1.5; 1.5], \quad \varepsilon = 0.01. \tag{1}$$

## Метод ломаных (Пиявского)

Функция называется Липшицевой на отрезке $[a, b]$, если существует константа $L > 0$ такая, что для любых $u, v \in [a, b]$ выполняется неравенство:

$$|W(u) - W(v)| \leq L \cdot |u - v|. \tag{2}$$

Для непрерывно дифференцируемых функций теоретическая константа Липшица равна максимуму модуля первой производной: $L \geq \max_{x\in[a,b]} |W'(x)|$.

В методе ломаных поиск глобального минимума осуществляется путём построения кусочно-линейной функции — миноранты $p(x) \leq W(x)$, составленной из пар лучей с наклонами $\pm L$, опущенных из верхних вершин $(u_i, W(u_i))$. Координаты нижних вершин (точек пересечения лучей $u_{cross}$) между соседними точками $u_q$ и $u_v$ рассчитываются по формулам:

$$u_{cross} = \frac{W(u_q) - W(u_v)}{2L} + \frac{u_q + u_v}{2}, \tag{3}$$

$$p(u_{cross}) = \frac{W(u_q) + W(u_v)}{2} - \frac{L(u_v - u_q)}{2}. \tag{4}$$

**Критерий останова:** Так как ломаная является минорантой, в каждой

новой точке испытания теоретически гарантировано выполнение неравенства $W(u_{new}) \geq p(u_{cross})$. Следовательно, разность обязана быть неотрицательной:

$$0 \leq W(u_{new}) - p(u_{cross}) \leq \varepsilon. \tag{5}$$

Использование абсолютной величины $|W(u_{new}) - p(u_{cross})|$ некорректно: ситуация $W(u_{new}) < p(u_{cross})$ свидетельствует о нарушении условия миноранты вследствие недооценки константы Липшица ($L < L_{true}$). В таком случае метод не может гарантировать нахождение решения с заданной точностью.

### Расчёт константы Липшица

Функция calculate_lipschitz строит сетку из 2000 точек и вычисляет максимальное численное отношение приращений dy / dx.

Оценка константы по конечной сетке точек не даёт строгой теоретической гарантии (между узлами сетки возможен узкий всплеск производной). Поэтому вводится эвристический коэффициент запаса 1.1, снижающий риск недооценки крутизны функции между узлами сетки:

```
def calculate_lipschitz(f, a, b, samples=2000):
    grid = np.linspace(a, b, samples)
    y_vals = np.array([f(pt) for pt in grid])
    dy = np.abs(np.diff(y_vals))
    dx = np.abs(np.diff(grid))
    l_calc = float(np.max(dy / dx) * 1.1)
    return max(l_calc, 1)
```

### Обработка граничных условий

В начале функции piyavsky_method строка формулы преобразуется в вызываемый объект через функцию eval с безопасным словарём функций. Проверяется вырожденность отрезка ($a = b$) с возвратом точного значения $W(a)$, при необходимости меняются местами границы ($a > b$), а при поиске максимума знак функции инвертируется ($W \to -W$):

```
    raw_f = lambda x: eval(func_str, {"__builtins__": {}}, {"x": x, "sin":
   np.sin, "cos": np.cos, "tan": np.tan, "exp": np.exp, "log": np.log, "
   sqrt": np.sqrt, "abs": np.abs, "pi": np.pi, "e": np.e})

    try:
        raw_f(a)
```

```
except Exception:
    print("\nНеправильно записана формула функции")
    return

if a == b:
    w_val = float(raw_f(a))
    print(f"\nГраницы совпадают. Отрезок равен точке: x* = {a}, W(x*) =
{w_val}")
    return a, w_val, 0, 0.0, True

if a > b:
    a, b = b, a

sign = -1 if mode == 'max' else 1
f = lambda x: sign * raw_f(x)
L = calculate_lipschitz(f, a, b)
```

**Алгоритм поиска минимума ломаной**

В цикле while верхние вершины сортируются по оси $x$, после чего рассчитываются координаты всех нижних вершин u_cross. Выбирается самая глубокая точка миноранты best_u, исключаются повторные вычисления в одной точке, вычисляется значение функции и проверяется односторонний критерий останова:

```
    new_u = best_u

    already_computed = False
    for idx, u_prev in enumerate(upper_u):
        if abs(new_u - u_prev) < 1e-9:
            new_w = upper_w[idx]
            already_computed = True
            break

    if not already_computed:
        new_w = float(f(new_u))

    diff = new_w - best_p

    if diff < -1e-6:
        lipschitz_broken = True
        upper_u.append(new_u)
```

```
            upper_w.append(new_w)
            break

        if diff <= eps:
            converged = True
            upper_u.append(new_u)
            upper_w.append(new_w)
            break

        upper_u.append(new_u)
        upper_w.append(new_w)
```

### Вывод данных и построение графика

После завершения расчётов числовые показатели выводятся в консоль вместе со статусом достижения точности, после чего вызывается функция plot_results:

```
    if lipschitz_broken:
        print("\nФункция оказалась ниже ломаной (константа L занижена, точно
сть не гарантирована)")
    elif not converged:
        print(f"\nДостигнут лимит в {max_iter} итераций (точность не гаранти
рована)")

    print(f"\nРежим оптимизации: {mode}")
    print(f"Отрезок: [{a}, {b}]")
    print(f"Константа Липшица L: {round(L, 4)}")
    print(f"Точка экстремума x*: {round(x_opt, 5)}")
    print(f"Значение функции W(x*): {round(w_opt, 5)}")
    print(f"Число проведенных итераций: {iterations}")
    print(f"Затраченное время: {exec_time:.5f} сек")

    plot_results(raw_f, a, b, upper_u, upper_w, L, x_opt, w_opt, func_str,
sign)

    return x_opt, w_opt, iterations, exec_time, converged
```

### Результаты экспериментов

Программа была запущена для тестовой многоэкстремальной функции Растригина. На рисунках 1 и 2 представлены численные результаты работы

программы в консоли и графическая иллюстрация найденного решения.

```
Режим оптимизации: min
Отрезок: [-1.5, 1.5]
Константа Липшица L: 71.865
Точка экстремума x*: 0.0
Значение функции W(x*): 0.0
Число проведенных итераций: 164
Затраченное время: 0.00945 сек
```

Рисунок 1 — Результаты работы программы в консоли

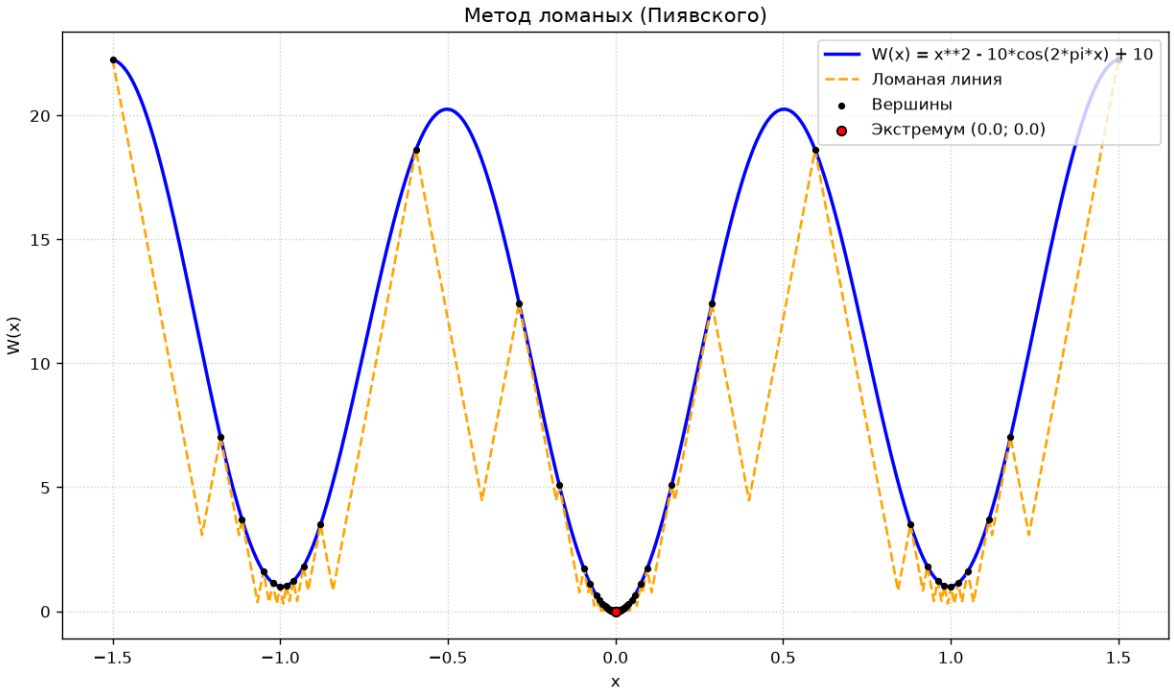



Рисунок 2 — График функции Растригина и построенная миноранта Пиявского

На графике видно, что алгоритм исследовал все три потенциальные ямы функции Растригина (в окрестностях $x \approx -1$, $x = 0$ и $x \approx 1$). Однако наибольшая плотность верхних вершин сконцентрировалась в окрестности центральной ямы, что привело к безошибочному нахождению истинного глобального минимума в точке $x^* = 0.0$ со значением $W(x^*) = 0.0$.

## Вывод

В ходе работы был изучен и реализован метод ломаных (Пиявского) для поиска глобального минимума функции на отрезке. Тестирование на функции Растригина подтвердило, что метод успешно находит истинный глобальный минимум и не застревает в локальных экстремумах.

В отчёте подробно рассмотрена роль константы Липшица: при её занижении функция пересекает миноранту сверху вниз ($W(u_{new}) < p(u_{cross})$), что делает невозможным гарантированный поиск оптимума. Результаты расчётов и построенный график ломаной линии наглядно подтвердили сходимость алгоритма и достижение заданной точности $\varepsilon = 0.01$.

## Ссылка на репозиторий

https://github.com/Marinasunshine/Optimization_Methods